

Transaction Agent 系统介绍

Business Overview | 业务简介

Background | 背景

在日常研发支持中，工程师频繁穿梭于工单、群聊、舆情反馈与问题排查之间：一边解释业务规则与口径，一边追踪链路、查看日志、核对指标、执行补偿。这些工作高度碎片化、重复性强且严重依赖个人经验，导致响应效率低、处理质量不稳定、新人上手困难。同理，针对TPS同学，每天要处理极其海量的ticket，导致工单有效解决率低，客户无法及时得到有效反馈。

因此，我们通过将一线高频问题抽象为两类核心能力形态（业务答疑与问题诊断），并结合“SOP技能化 + 质量评分闭环”机制，实现解释与排查工作的前置自动化。该系统不仅“能跑”，更能持续迭代进化，显著缩短首响时间与平均解决时长，提升服务一致性与工程效能。

Design Goals | 设计目标

业务目标：

在不增加一线心智负担的前提下，提供“从问题到结论”的标准化辅助：聚焦提升排障效率、降低误操作风险。支持两种核心工作模式：**排查诊断**（基于工具与技能的可执行链路）与**知识问答**（基于知识库与检索的解释型答案）

针对TPS：针对咨询排查类问题(非系统问题) 均能通过Agent 来辅助回答。

针对RD/PM：能够利用Agent协助排查相关问题。

系统目标：

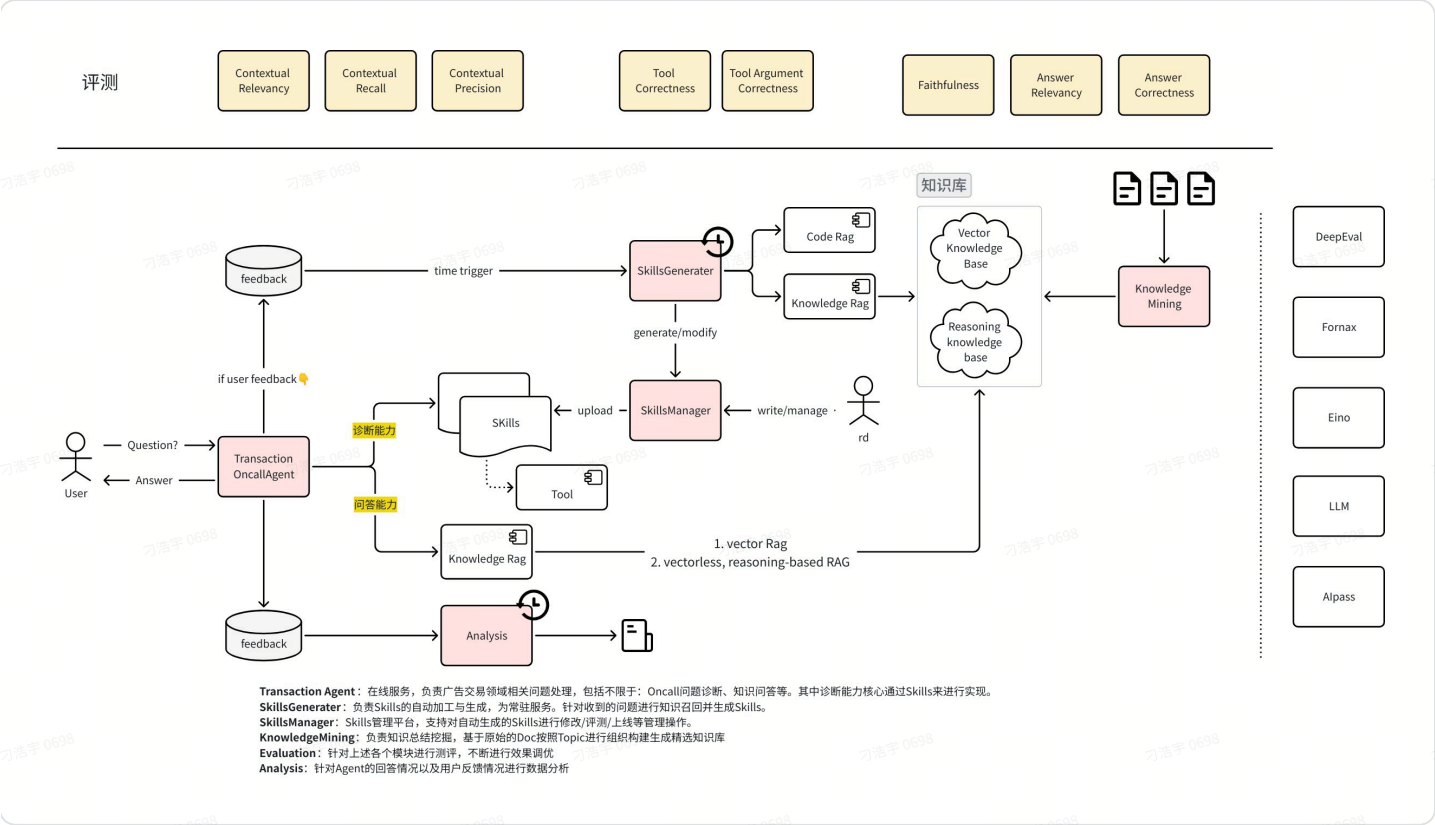
- 通过统一的“任务与事件流”抽象，兼容同步回复与流式回复，并为上游（OnCall/飞书）提供一致的对接方式。
- 配置与能力可热更新：Prompt、LLM、MCP 工具、路由策略等通过配置中心管理，降低发布频率。
- 可扩展与可治理：技能可版本化、可灰度、可回滚；输出可复盘并持续沉淀为 SOP / 自动化能力。

GTM Plan | 推广规划

Phase1	2026/04/01：完成Transaction Assistant 新feature发布，支持诊断能力上线 Transaction Assistant 新Feature发布（2026/03/31）
Phase2	2026/06/01：完成Transaction Assistant 接入Athena

Overall Design | 整体设计

System Architecture | 系统架构



我们将整体系统分为以下5个核心部分

核心模块	相关说明
OncallAgent	对外提供知识问答、问题诊断 两个核心能力，支持支持同步与流式两种模式。
SKillsManager	针对生成的/编写的SKill 进行版本管理和上下线，并支持debug调试能力； 同时支持针对不同场景加载不同的SKill，做到同一份SKill 多场景应用。
SKillsGenerater（建设中）	基于feedBack离线SKill加工自动生成，主要为了生产SKill，供OncallAgent进行使用
Knowledge Mining（建设中）	针对交易领域相关的文档/代码等知识进行采集和分析挖掘，生成高质量的知识供OncallAgent/SKillsGenerater来进行使用

User Scenario | 用户示例

在下面的群众@机器人，或者直接私聊“transaction Assistant” 机器人进行使用，具体可参考[Transaction Assistant 新Feature发布（2026/03/31）](#)



Transaction Assistant

Transaction Assistant

群名片

诊断能力

Transaction Assistant

我是一个集成智能问答、人工支持和智能分析的交互式服务平台。智能问答：输入问题，查看详情。机器人：请和我聊聊。

coupon_usage_condition

4 (具体含义业务枚举定义，一般用于描述使用条件类型)

3/5

基础信息查看情况 (按 Skill 要求)

基本信息：已包含 coupon_code, coupon_id, project_business_type, coupon_category (券类型), coupon_status (券状态), 面额/折扣 (coupon_business_amount / coupon_discount_amount), 活动 id (coupon_activity_id), capital 相关字段 activity/project 间接体现。

有效期与时间：使用 issue_time, verificate_start_time, verificate_end_time, verificate_time 均已转换为 UTC 标准格式。

使用门槛：min_spend_amount = 0, 无最低消费门槛。

使用/核销条件：本工具返回中仅有 coupon_usage_condition 的枚举值，没有更详细的使用条件与核销条件结构字段；如需查看更细的使用条件或核销条件，需要进一步用 coupon_mcp_server_multi_coupon_detail 以 coupon_id 为维度查询。

处理建议

如果你只需要“基础信息” (券状态、金额、有效期、活动与广告主归属等)，可以直接参考上表使用。

如果你还需要：

更具体的“使用条件” (如可用商户、可用场景、是否限定投放目标等)，或

更细粒度的“核销条件” (如核销规则分段、组合条件等)。

按 Skill 指引的下一步，可使用 coupon_mcp_server_multi_coupon_detail，以 coupon_id=7499482279589478673, project_business_type=42 进行查询，从返回的 deduction_coupon_detail.deduct_condition_info 与 common_detail.condition_detail 中获取完整规则。

如果你有指定的业务侧账号或活动信息，也可以补充给我，我可以帮你进一步解释该券在你具体投放场景中的可用性与实际影响。

Used Skills: Skill-coupon-info-query

Selected Skills: Skill-coupon-info-query

Atomic Tools: batch_get_coupon_by_coupon_codes, coupon_mcp_server_multi_coupon_detail

Resolved

Unresolved

Feedback won't affect oncall, feel free to feedback.

15:10

/clear

Context cleared successfully

备注

沟通时请保持“公开可搜索”

00:00:00 / 01:00:00

结束录制

问答能力

Transaction Assistant

我是一个集成智能问答、人工支持和智能分析的交互式服务平台。智能问答：输入问题，查看详情。机器人：请和我聊聊。

deal_no: 空

split_has_coins: 0

split_subject_id: 0

business_line2: 1

split_month: 202601

split_id: 0

biz_order_type: -98

唯一差异字段：

billing_group_id:

ST7598003691060855047 -> 7573546828126890912

ST7598003691060871431 -> 7595161767619121426

出账时，系统会按多维度 group by (结算客户、币种、business_line, settlement_pay_method, pricing_category, biz_order_id, deal_no, billing_group_id 等) 生成对账单。

当 billing_group_id 不同时，即便其他维度都相同，也必须拆成不同的对账单。

这说明：

该客户在同一账期内，存在两个不同 Billing Group 维度的消耗；

系统按 BG 粒度拆单，所以生成了两张账单，而不是“本来只应该一张却生成两张”的错误。

处理建议

对当前结果的解释

当前这两张账单：

账期相同 (split_month=202601)，出账时机类型相同 (bill_type=1)，支付方式相同 (MI) 且都是 Auction 消耗；

差异点仅在于 billing_group_id 不同，符合“按 BG 粒度拆单”的出账规则；

因此，“一个月一张账单”这句话需要理解为：同一个出账口径/同一拆分维度通常是一个月一张账单；但如果存在多个 Billing Group 或其他拆分维度差异，就会在同一个月产生多张账单，这是系统设计预期。

如需进一步确认/优化

若业务方希望“一个月只看到一张账单”，需要：

统一调整该结算客户在信用/出账配置中的 Billing Group (billing_group_id)，确认是否可以合并；

或在内部与配置相关团队 (BG 配置 / CL 配置) 确认这两个 billing_group_id 的业务含义 (例如对应不同产品线/组别)，看是否有必要继续拆分出账。

如果你再提供具体的账期 (例如明确是 2026-01) 以及期望的“出一张账单”的口径 (按客户合并? 按 BG 合并?)，我可以帮你进一步梳理应该如何配置来达到该效果。

Used Skills: Skill-recon-sql-query, Skill-recon-062-generate-reason-issue-report

Selected Skills: Skill-recon-062-generate-reason-issue-report, Skill-recon-sql-query

Atomic Tools: crux_common_query, cmd_tool

Resolved

Unresolved

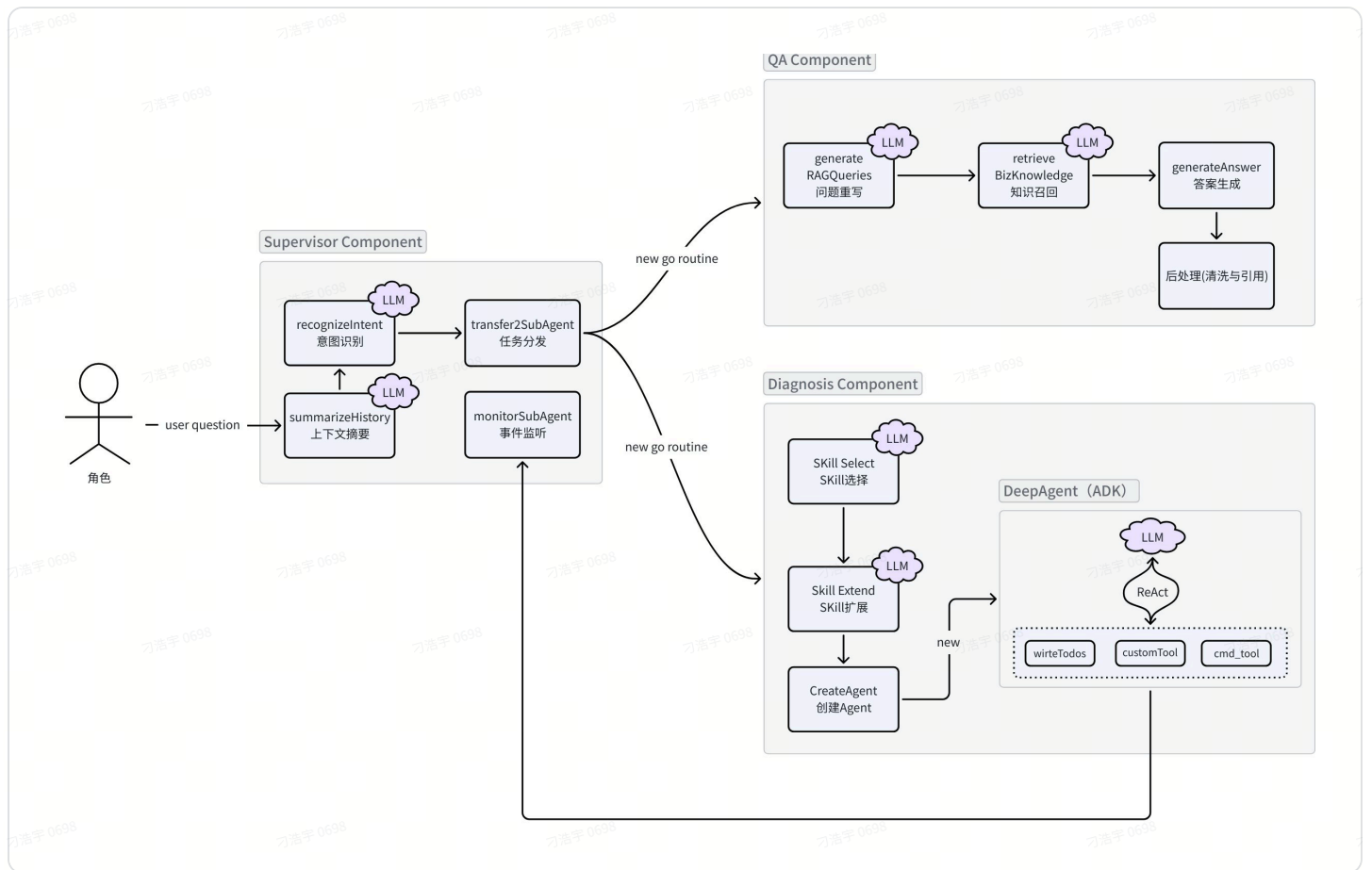
Feedback won't affect oncall, feel free to feedback.

00:00:00 / 01:00:00

结束录制

Overall Process Flow | 整体处理流程

OncallAgent



OncallAgent 是一个基于大语言模型（LLM）的智能体系统，主要用于处理用户的 Oncall 问答、业务诊断、知识检索以及工具调用。系统采用了典型的多智能体协同架构（Multi-Agent Architecture），能够根据用户的意图自动路由到最适合的子智能体进行处理，并通过流式事件（SSE）将推理过程和最终结果实时返回给用户。

Diagnosis Component | 诊断能力介绍

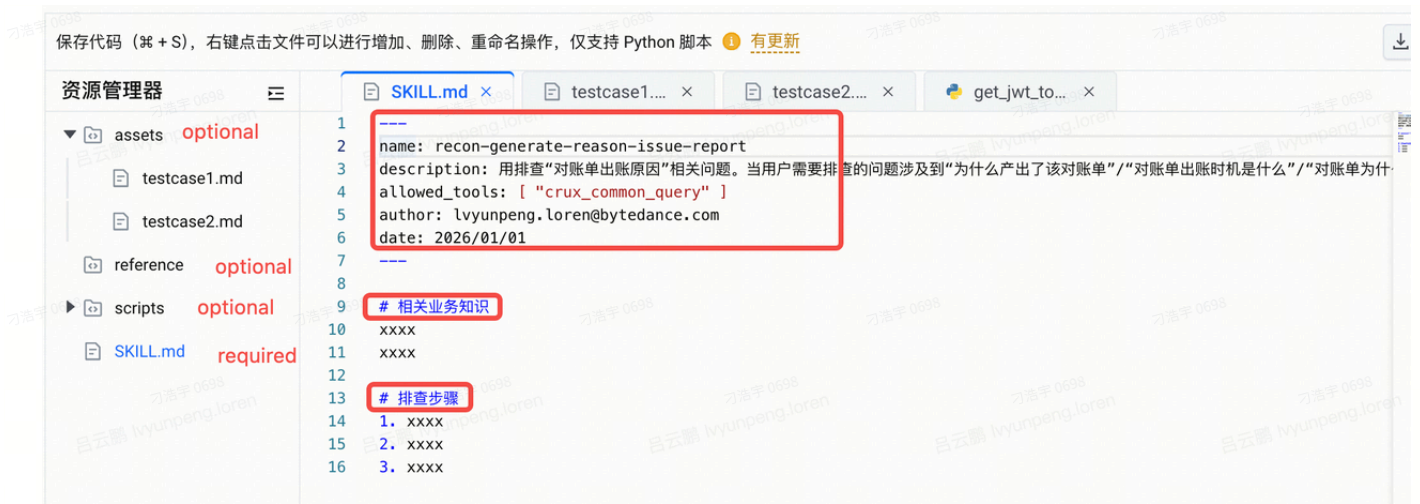
关于诊断能力的构建，我们进行了多种版本的尝试与构建思路，最终演变成当前以Skills为基石的架构。

Ability1：初尝Skill-实现基于Skill的诊断能力

2025-10-16 [anthropics](#) 官方首次对外介绍 *Claude Skills*能力，并于12月将 Skills 以 GitHub repo 形式公开

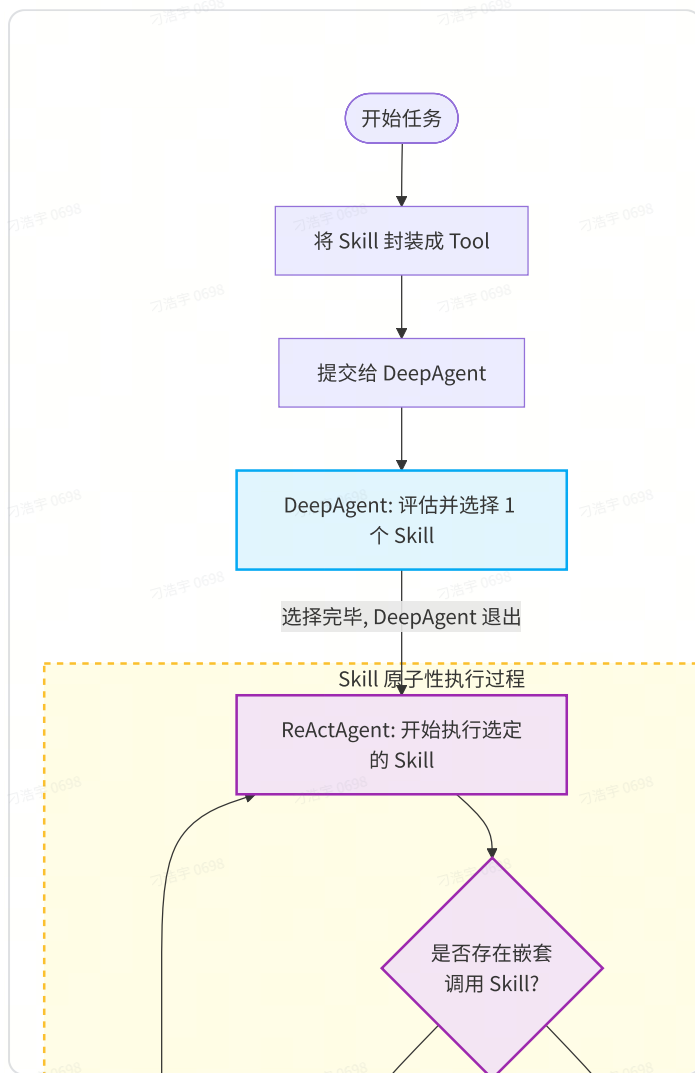
Skill定义

参考<https://agentskills.io/home>，制定Skill的格式如下：[Transaction OncallAgent Skill规范](#)，我们将一个一个的问题解决Sop 给抽象成一个个具体的Skill，然后交给Agent进行执行。



Skill目录结构示例

SKILL执行



最终效果

在这个版本，我们已经有一个执行相对稳定的诊断能力，并开始推动整个交易来进行SKills的初期加工和收集，开始在生产环境小范围使用了。而在进一步推广和使用中，我们遇到了下面几个问题：

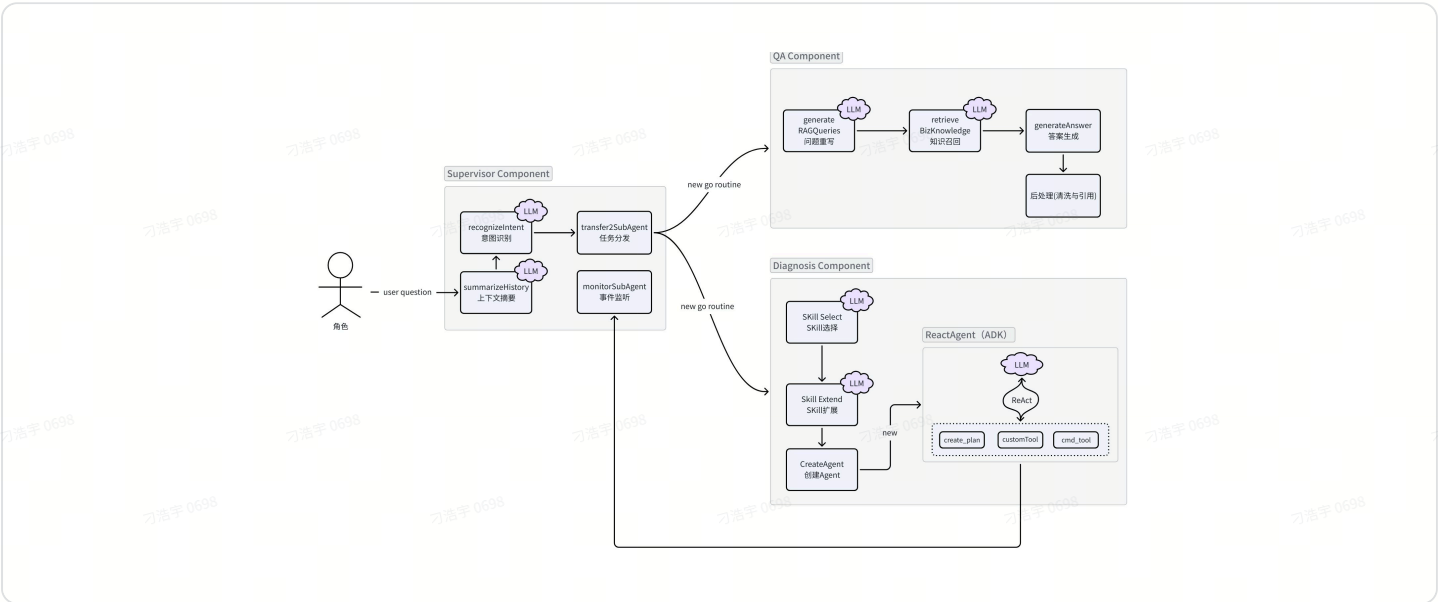
耗时问题：当前支持Skill调用Skill(要求Skill显示指定被调用Skill的Name)，但是实现机制是将每一个Skill都封装成一个Mini-Agent(ReAct)，这种嵌套调用比较耗时，希望更进一步优化

复杂问题解决：当前模式针对用户问题，只能选择一个Skill来解决。不能针对用户的问题来进行编排灵活的调用多个Skill来执行。比如我有一个Skill是解释对账单出账规则，还有一个Skill是解释发票和对账单的关系。用户询问为什么这个月给我出了2张发票，那么就需要将这两个Skill进行组合后才能够回答。

跨域问题解决：当前各个领域的Skill为每个方向自己实现，比如对账单同学写了一个skill，其中有一个处理步骤写的是“我需要去查询一下这个账单上资金当时是如何充进来”，然后拿到当时充值时刻的主体信息来判断当前对账单上的主体是不是不符合预期。针对这个子问题，对账单同学没有办法也不知道如何去获取。因此我们需要能够支持在执行某个Skill时，针对里面的某个子问题，再次去寻找合适的Skill去解决。

Ability2: Skill编排能力建设

核心思路



在当前方案中，我们不再将一个Skill抽象成一个Tool注册给Agent，也不需要为了解决Eino Agent动态加载Tool的问题，来将一个Skill的执行包装成一个独立ReActAgent。

- 技能路由 (Skill Router)：基于 skill_router中的策略（支持单阶段、基于领域的二阶段 meta_tool、或者直接的 tool_call），利用大模型从技能库中挑选出最适合解决当前问题的技能 (Skills)。
- 依赖解析 (Dependency Resolution)：通过 BFS 算法解析被选技能的依赖关系。技能可以依赖原子工具 (Atomic Tools)，也可以包含占位符（{{子任务}}）去递归调用其他技能，最终生成一张技能执行图 (Skill Graph)。
- ReAct 执行 (ReAct Executor)：将技能图、技能说明以及用户问题组装成 System Prompt，交由 react_executor 处理。底层基于 Eino 框架的 ChatModelAgent 驱动，Agent 会在循环中不断“思考 -> 调用工具 -> 观察结果”，直到解决问题。

Skill选择

我们将Skill选择和Skill的执行 拆分成两个不同的环节； 在Skill选择环节，通过问题分析与匹配等方式，先给出一批可能解决该问题的Skill 候选集； 然后在Skill执行阶段再根据这些候选集合进行Plan组合去最终执行解决用户的问题。

Skill执行

在进行执行时，并不会直接将选择的整个Skill 包 传递给上下文，而是只会先将这个Skill的 Description传递给LLM，由模型使用cmd_tool 去根据实际情况选择性阅读对应的文件。

1. 什么是 Skill?

在你的认知体系中，Skill 是解决某一类特定故障的“标准化诊断与处理包”。

- **它是封装好的专家经验**：每个 Skill 都浓缩了资深工程师解决特定系统、特定组件故障的最佳实践。

- **它是可执行的实体**：它不是一份被动阅读的说明书，而是包含触发条件、诊断步骤 (SOP)、工具和输入/输出接口的执行体。

- **它是你的能力边界**：你的排查工作必须基于具体的 Skill 展开。面对问题，你的第一反应永远是“我应该调用哪个 Skill”，而不是毫无根据地自由发挥。

|

2. Skill 的标准物理结构 (The Anatomy of a Skill)

在你的运行环境中，每个 Skill 都以特定文件集的形式存在。当你挂载一个 Skill 时，你必须清晰地知道去哪里获取诊断信息：

- *** Skill.md (主控文件/核心医嘱) ***
 - * 这是每个 Skill 的灵魂与唯一入口。
 - * **适用场景 (Scenarios)**：定义了该 Skill 解决什么具体问题。这是你进行“Skill 路由匹配”的绝对依据。
 - * **标准操作流程 (SOP)**：***所有的排查步骤都严格且唯一地定义在这里***。这是你的“诊断检查表”，你必须逐行阅读Skill.md中的“排查步骤”进行排查，绝不可跳过或自行编造步骤。
 - * **references/ 文件夹 (辅助资源库/工具箱)**：***
 - * 这是一个可选的旁路资源目录。它存放了 SOP 落地所需的基础材料，例如：特定 Tool 的详细使用说明、API 请求的 JSON 示例、复杂的 SQL/Python 脚本、或日志数据文件等。
 - * ***⚠️ 触发查阅原则***：你不能预先假设或捏造“references/”中的任何内容。***只有当“Skill.md”的排查步骤中明确要求、或暗示你需要参考某个配置文件/请求示例/脚本时***，你必须立即停止主观猜测，主动调用你的文件读取工具（如“cmd_tool”）去查

作为高级诊断 Agent，在本次任务中你可以调度以下 Skill 来完成排查。请在【分析与路由】阶段仔细阅读本节内容，并在生成【执行计划】时充分考虑 Skill 之间的调用关系。

1. Skill 调用与依赖关系 (Dependency Graph)

部分复杂的“Skill”在执行时可能需要依赖或触发“子 Skill”来获取特定信息/执行特定动作，下面是Skill之间的调用与依赖关系图。

- **格式说明**：~ Skill-A调用了: Skill-B, Skill-C

- **编排要求**：当你决定将某个 Skill(或者某几个Skill) 作为切入点时，必须评估是否需要将其包含的子 Skill 纳入你的执行计划中，以保证排查链路的完整性。

<Dependency_Graph>
{ {SKILL_GRAPH}}
</Dependency_Graph>

2. 可用 Skill 详情列表 (Available Skills)

一旦决定使用某个 Skill，**你必须使用命令行工具“cmd_tool”读取其对应的“Skill.md”文件中的内容**，以获取具体的 SOP 步骤。如果有必要或者“Skill.md”中有要求，请查看“references/”目录下的文件。绝对不得凭空捏造文件内容、SOP 步骤或执行结果。

<Skills_List>
{ {SKILL_SOPS}}
</Skills_List>

三、排查与执行流程 (Execution Workflow)

作为高级诊断 Agent，你的执行过程必须高度严谨、基于事实，并严格遵循“先规划，后按规执行”的模式。你的完整工作流如下：

Step 1. 核心路由与生成计划 (必须调用 create_plan 工具)

在收到用户问题后，不要立即开始直接排查或回答。你必须首先全面评估用户需求与 Available Skills 的匹配度，并通过调用“create_plan”工具生成并输出一份明确的排查计划。

规划内容要求：明确列出需要调用的主 Skill、预期的执行步骤，以及要解决的具体子问题。

注意：调用“create_plan”后，系统会将该计划保存并注入到你的执行上下文中，作为后续排查的基准。

Step 2. 严格按规执行 (Execution Rules)

<must_follow>
[红线 1] 严格遵循 SOP：你必须按部就班地读取目标 Skill 目录下的 Skill.md 文件。必须严格按照文档中定义的排查步骤、前置条件和顺序进行操作。[严禁] 跳过任何关键步骤。[严禁] 在没有事实依据的情况下自行猜测结论。
</must_follow>

<must_follow>
[红线 2] 强制使用 cmd_tool 访问本地系统：所有对文件系统的访问（如读取 Skill.md、查看日志、读取 SQL 等），必须且只能通过调用 cmd_tool 完成。[严禁] 捏造未实际读取的文件内容或未执行的系统状态。

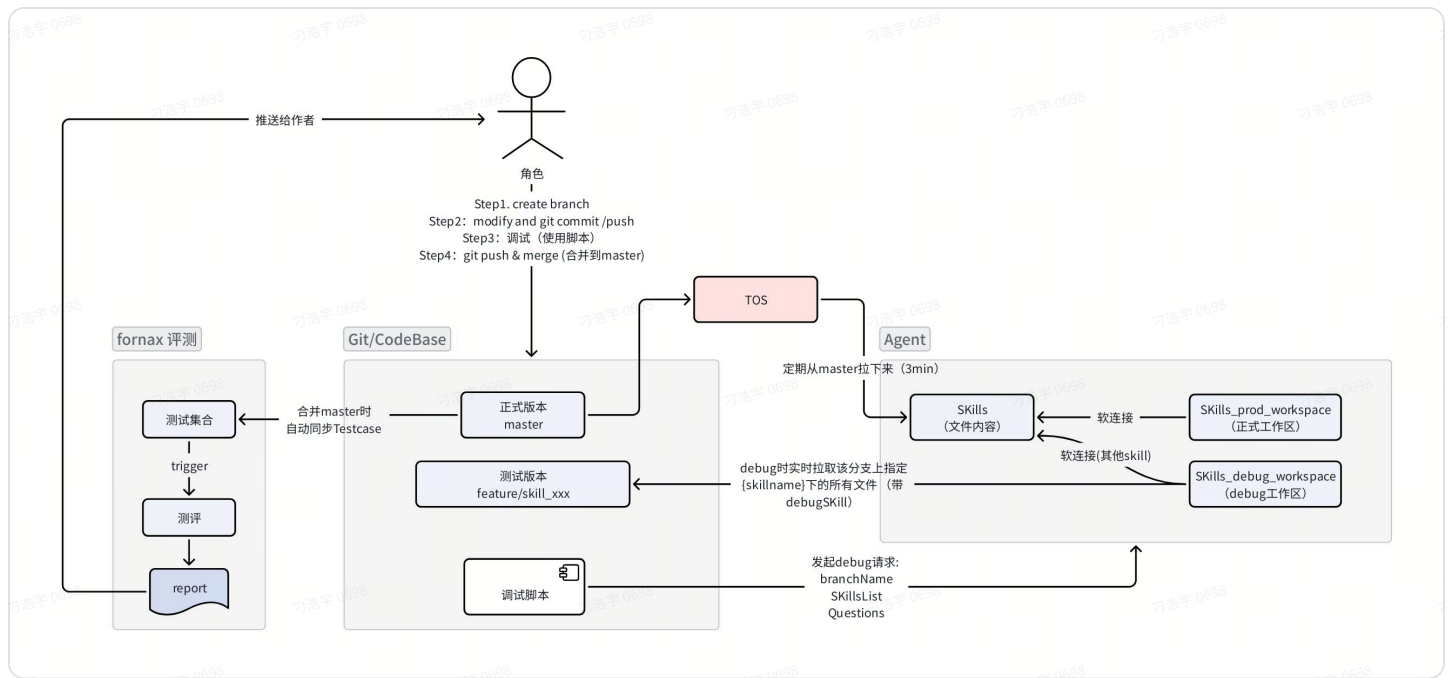
Ability3: Skill版本管理与Debug

在 OncallAgent 的实践中，我们遵循官方定义的 Skill 规范，构建了一套基于本地文件系统的 Skill 体系。这套体系能满足基本需求，但也逐渐暴露出一些核心痛点：

测试与版本管理困难： 当需要调试一个新版本的 Skill (例如 Skill-A-v2) 或希望在一个测试环境中验证 Skill-A-v2 调用 Skill-B-v3 这样的跨版本依赖时，现有文件结构难以支持。直接修改会污染线上环境，而为每个测试组合 Fork 整个 Skill 库又会导致磁盘空间冗余和管理混乱。

核心思路

为了解决测试文件污染和版本管理混乱的问题，我们引入了 MVCC-Skill方案。其核心思想是“解耦 Skill的 '存储' 与 '运行时环境'”，通过 软链接(Symbolic Link) 实现多版本并存、快速切换和环境隔离。



在新版本Skills实现里，每个Pod都会上有这几个文件夹：

- **Skill Repo (技能仓库)**：一个包含“正式版本” Skill文件夹。这是所有 Skill 文件的真实存储位置。
- **Skill Published (已发布版本文件夹)**：一个专门用于处理线上流量的Skill目录。该目录下的内容全部是**软链接**，分别指向 Skill Repo 中对应的**线上已发布版本**。
- **Skill Tested {skill-name}-{uniqueId} (测试环境)**：当需要调试特定 Skill 时，我们会创建一个独立的测试目录。该目录同样由软链接构成，其中待测试的 Skill 指向 Skill Repo 中的测试版本，而其他依赖的 Skill 则指向稳定版本或指定的其他版本。

该方案有如下优点：

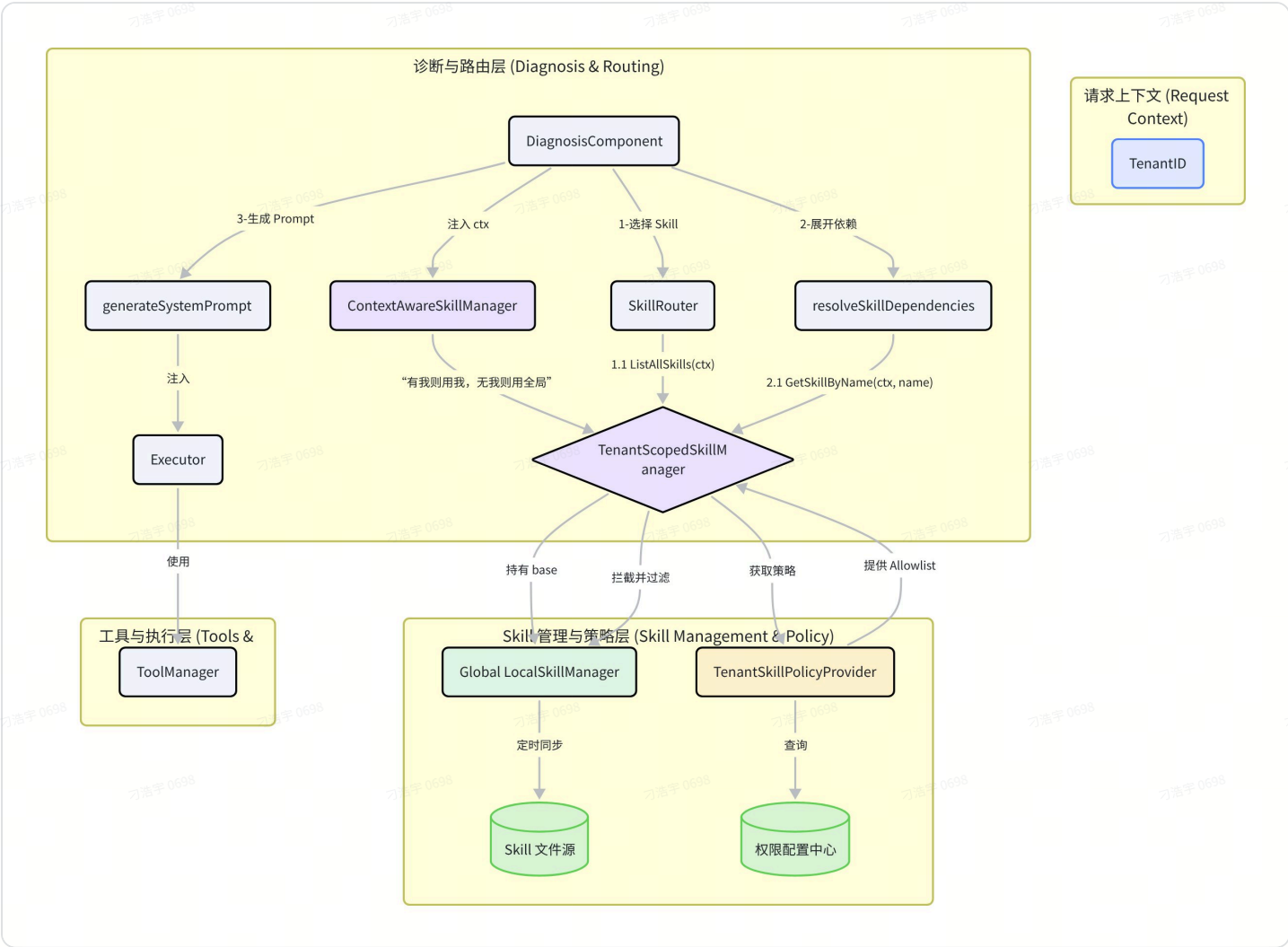
- **轻量级**：创建软链接几乎没有磁盘开销（仅几字节），创建速度极快，相比于拷贝整个文件夹的方案，性能优势巨大。
- **环境隔离**：测试环境和生产环境完全分离。测试用的 Skill 版本不会污染 Published 目录，Agent 在各自环境中只会看到其应该看到的文件。
- **快速切换与回滚**：无论是版本发布还是故障回滚，都只需修改软链接的指向即可秒级完成，极大提升了运维效率和系统的稳定性。
- **保留结构**：该方案完美保留了原始文件系统的组织结构，便于理解和维护。而且和之前Skill存储在Git上的原始方案，完全兼容。

Ability4: SKill多租户管理与自定义输出能力

核心思路

最初的SKill 只是给TA 这一个场景使用的，之后随着TA 要接入雅典娜、OCIC等多个平台，不同的租户对TA 的SKill 可见范围和回答方式不一样，因此我们要针对不同的租户支持不同的SKill能力范围以及输出格式提供。

简单来讲：请求时会携带租户名称，会根据租户来进行Skill的过滤以及prompt的选择等



QA Component | 问答能力演进

Ability1: ByteRag+自建知识库

核心思路

在这个阶段，我们主要解决知识问答本身链路的搭建，采用自建交易知识库的方式来实现（将指定目录下的飞书doc 自动化同步到Byterag中）

本身的流程为：问题重写(子问题拆解)->并发多路召回->知识总结->结构化输出等

最终效果

能够较好的回答交易领域的相关知识，但是回答的内容比较偏RD，同时针对一些跨平台的问题没有办法较好的回答，因为没有对应的知识。

Ability2: 用户分群+引入SalesAlly知识库

核心思路

知识库新增：除了自身RD相关的知识（技术文档/需求文档/codebase）等，引入了SalesAlly的知识库（具备各个平台的help doc 以及业务的common doc），使得交易的知识更加全面。

用户分群 (User Segmentation)：提取上下文与用户信息，系统会根据用户名匹配该用户所属的分群（Segment）。不同的用户群体在后续检索时，可能会应用不同的权重和召回数量（TopK）限制。

知识总结：分别从两个知识库进行分片召回，并按照权重进行组合排序，总结返回给用户。

最终效果

兼顾了普适性问题以及技术性问题的回答，回答准确率及效果有所提升

Evaluation Component | 评测能力演进

Ability1：诊断场景评测能力建设

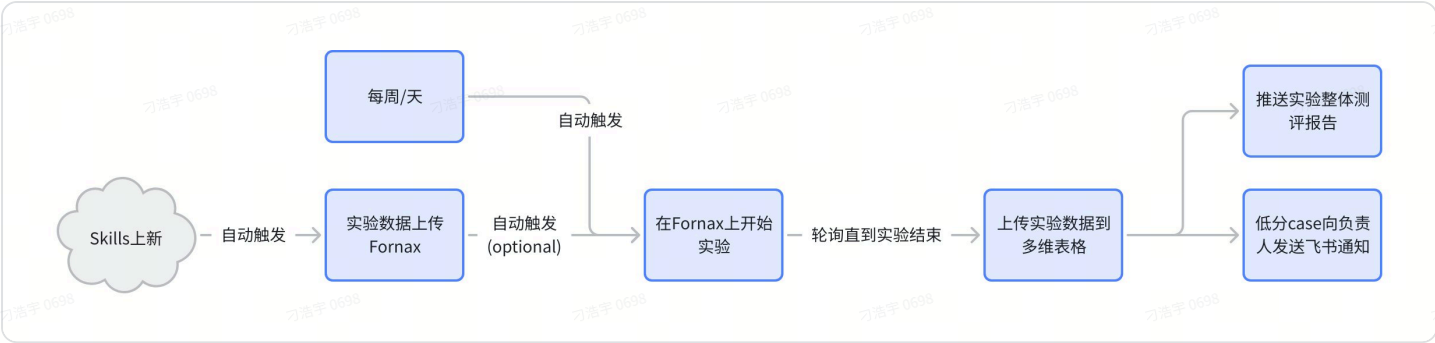
核心思路

以自动化测试（LLM-as-a-Judge）为主，定期人工检查作为验证。开发阶段通过分析失败测试用例和LLM打分和人工打分的一致性来迭代LLM打分的Prompts和测试集用例。评估器采用Eval框架实现，并集成在Fornax第三方评估器

相关指标

指标	描述	输入	计算方法
Skill Select Correctness (核心指标)	Skill 选择准确率	actual_output 和 expected_skill	使用LLM提取出 actual_output 中所有的Skill Name，和 expected_skill 比对，统计一致的比率
Answer Correctness (核心指标)	直接与参考答案对比，判断是否一致	actual_output 和 expected_answer	使用LLM判断实际输出 actual_output 和参考答案 expected_answer 在事实和逻辑上是否一致
Answer Relevancy (次生指标)	和问题相关性评估	actual_output 和 question	使用LLM提取 actual_output 中所有的命题（statements），然后统计和 question 相关的命题数，除以总命题数

最终效果



- 1. 每次实验结果会记录在 [Agent Evaluation Results - Original Data](#)，同时生成推送信息给到作者
- 2. 每次实验会生成测评报告，详情见：<https://exp-20260324-020135-7590053957076328708.gf-preview.bytedance.net/>

Ability2: 问答场景评测能力建设

相关指标

指标	描述	输入	计算方法
Answer Relevancy	和问题相关性评估	<code>actual_output</code> 和 <code>question</code>	使用LLM提取 <code>actual_output</code> 中所有的命题（statements），然后统计和 <code>question</code> 相关的命题数，除以总命题数
Contextual Relevancy	召回的文本中有多少是和问题相关的	<code>retrieved_chunks</code> 和 <code>question</code>	这里我们不将整个chunk作为一个整体，而是利用LLM将每个chunk分解成命题（Statements），以此为粒度进行计算 先使用LLM先提取出 <code>retrieved_chunks</code> 所有的命题并判断是否与问题相关，然后用相关的命题数除以总命题数
Faithfulness	可信度评估	<code>actual_output</code> 和 <code>retrieved_chunks</code>	使用LLM提取出 <code>actual_output</code> 中所有的断言（claims），然后统计有 <code>retrieved_chunks</code> 支撑的断言数量，除以总断言数

核心思路

直接回流线上的真实问题，对齐输出进行打分评测，来实际评估回答的准确性。

Conversational Skill Generation (Local)

在最初的版本，我们的Skill 完全是人工来进行编写，除了需要关注业务逻辑本身，还需要关注如何将Skill写的足够稳定和健壮，这个成本是相当高的。在未能完全自动生成Skill之前，我们提供了对话式编写Skill的能力，来让用户只关心Skill的逻辑即可。📖 [如何快速编写一个Skill?](#)

具体实现为：针对不同场景，提供标准的Skill_Template模版

- **skill-writer**: 利用claudecode的create_skill技能，结合Skill_Template模版，通过和用户对话来生成Skill
- **skill-quality-audit**: 检测生成的Skill是否符合Template里面定义的lint规则，不符合则进行优化修改

具体流程：

实际效果：

代码块

```
1  你：帮我创建一个 skill
2      ↓ Claude 提问，你回答四个问题
3  你：（回答 skill 目标、触发场景、工具、输出）
4      ↓ Claude 阅读现有 skill + 规范，生成初稿
5  你：（审核初稿，指出问题）
6      ↓ Claude 修改
7  你：（贴入 IDL / 业务文档 / 补充业务逻辑）
8      ↓ Claude 整合业务知识
9  你：跑一下质量检测
10     ↓ Claude 审计 + 自动修复
11  你：还有一个遗漏的业务逻辑...
12     ↓ Claude 补充
13  你：提交到远端，测试一下
14     ↓ Claude 提交 + 启动测试
```



Semi-automated Skill Generation (with Oncall Process)

除了本地直接编写Skill，我们可以将Skill的编写和Oncall的处理形成联动，当Oncall结束的时候，会推荐用户是否需要生成/优化Skill，如果需要，那么拉起SkillGenerator，基于Oncall去生成TestCase(问题和期望输出)、以及复用上述对话式生成Skill的模式，通过和Oncall处理人的协作，来生成一个可以解决此类问题的Oncall，并自动运行Skill质量检测等功能，所有事项都通过后，生成一个Skill的正式上线的MR，供review合入。

Knowledge Mining

规划中...

Reference | 参考